

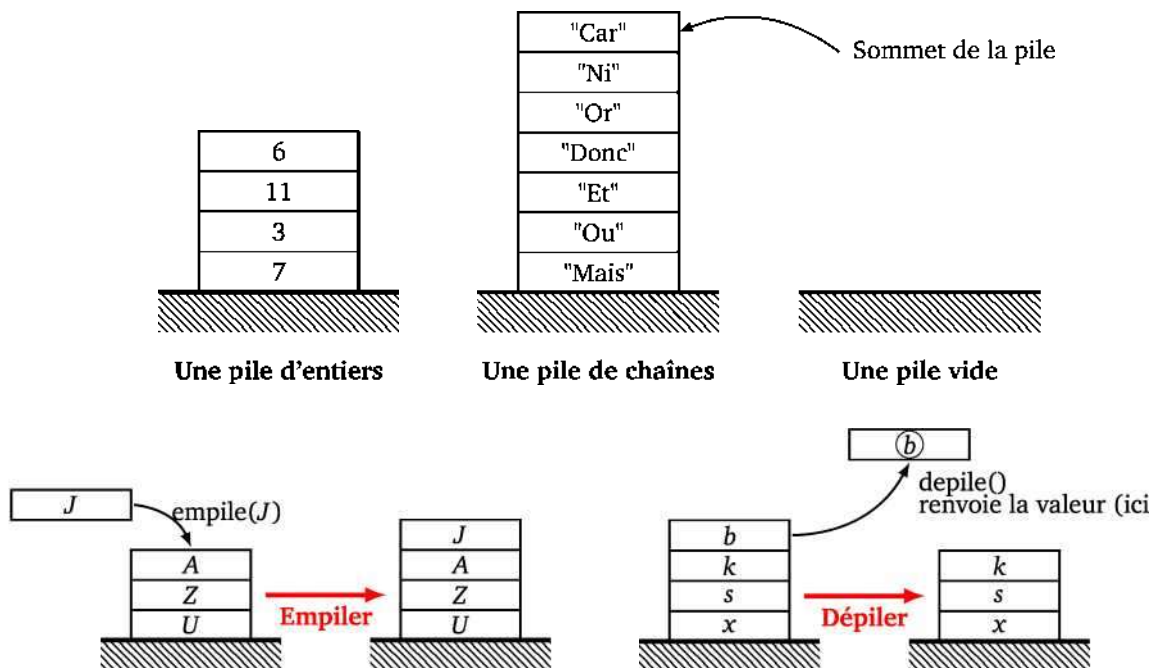
Calculatrice polonaise

Un exemple d'utilisation de la notion de pile en informatique

Cours 1 (Pile).

Une *pile* est une suite de données munie de trois opérations de base :

- *empiler* : on ajoute un élément au sommet de la pile,
- *dépiler* : on lit la valeur de l'élément au sommet de la pile et on retire cet élément de la pile,
- et enfin, on peut tester si la pile est vide.



Remarques.

- **Analogie.** Tu peux faire le lien avec une pile d'assiettes. On peut déposer, une à une, des assiettes sur une pile. On peut retirer, une à une, les assiettes en commençant bien sûr par celle du haut. En plus, il faut considérer que sur chaque assiette est dessinée une donnée (un nombre, un caractère, une chaîne...).
- **Dernier entré, premier sorti.** Dans une file d'attente, le premier qui attend, est le premier qui est servi et ressort. Ici c'est le contraire ! Une pile fonctionne selon le principe « dernier entré, premier sorti ».

- Dans une liste, on peut accéder directement à n'importe quel élément ; dans une pile on n'accède directement qu'à l'élément au sommet de la pile. **Pour accéder aux autres éléments, il faut dépiler plusieurs fois.**
- Avantage d'une pile : c'est une structure de données très simple qui correspond bien à ce qu'il se passe dans la mémoire d'un ordinateur.

Activité 1 (Opérations sur la pile).

Dans cet exercice, on travaille avec une pile formée d'entiers. Les questions sont indépendantes.

1. En partant d'une pile vide `p`, écrire la suite d'instructions qui modifie la pile `p` pour obtenir le contenu 5, 7, 2, 4, dans cet ordre, 5 étant le premier empilé et 4 le dernier, donc le sommet de la pile.
2. Écrire une fonction `pile_contient` qui prend en paramètres une pile et un élément ; cette fonction teste si la pile contient cet élément en renvoyant `True` si l'élément est dans la pile, `False` sinon. Le code de la fonction devra faire appel à la fonction `depile`.
3. Écrire une fonction `calcule_somme` qui prend en paramètre une pile, calcule et renvoie la somme des éléments de la pile.
4. Écrire une fonction `avant_dernier` qui prend en paramètre une pile et renvoie l'avant-dernier élément de la pile s'il existe, `None` sinon.

Cours (Manipulation de chaînes).

1. La fonction `split()` est une méthode Python qui sépare une chaîne de caractères en morceaux. Si aucun séparateur n'est précisé, le séparateur est le caractère espace.

python : `split()`

Usage : `chaine.split(separateur)`

Entrée : une chaîne de caractères `chaine` et éventuellement un séparateur `separateur`

Sortie : une liste de chaînes de caractères

Exemple :

- `"Etre ou ne pas etre.".split()` renvoie `['Etre', 'ou', 'ne', 'pas', 'etre.']`
- `"12.5;17.5;18".split(";")` renvoie `['12.5', '17.5', '18']`

2. La fonction `join()` est une méthode Python qui recolle une liste de chaînes en une seule chaîne. C'est l'opération inverse de `split()`.

python : `join()`

Usage : `separateur.join(liste)`

Entrée : une liste de chaînes de caractères `liste` et un séparateur `separateur`

Sortie : une chaîne de caractères

Exemple :

- `"".join(["Etre", "ou", "ne", "pas", "etre."])` renvoie `'Etreounepasetre.'` Il manque les espaces.
- `" ".join(["Etre", "ou", "ne", "pas", "etre."])` renvoie `'Etre ou ne pas etre.'` C'est mieux lorsque le séparateur est une espace.
- `"--".join(["Etre", "ou", "ne", "pas", "etre."])` renvoie `'Etre--ou--ne--pas--etre.'`

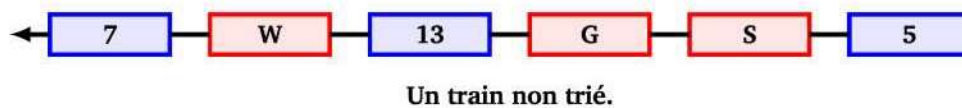
3. La fonction `isdigit` est une méthode Python qui teste si une chaîne de caractères ne contient que des chiffres. Cela permet donc de tester si une chaîne correspond à un entier positif. Voici des exemples : `"1789".isdigit()` renvoie `True` ; `"Coucou".isdigit()` renvoie `False`. Rappelons que l'on peut convertir une chaîne en un entier par la commande `int(chaine)`. Le petit programme suivant teste si une chaîne peut être convertie en un entier positif :

```
machaine = "1789"                # Une chaîne
if machaine.isdigit():
    monentier = int(machaine)      # monentier est un entier
else:
    # Problème
    print("Je ne sais pas convertir la chaîne en un entier ! ")
```

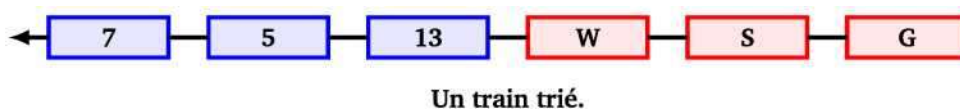
Activité 2 : (Gare de triage).

Objectif : résoudre un problème de triage en modélisant une zone de stockage par une pile.

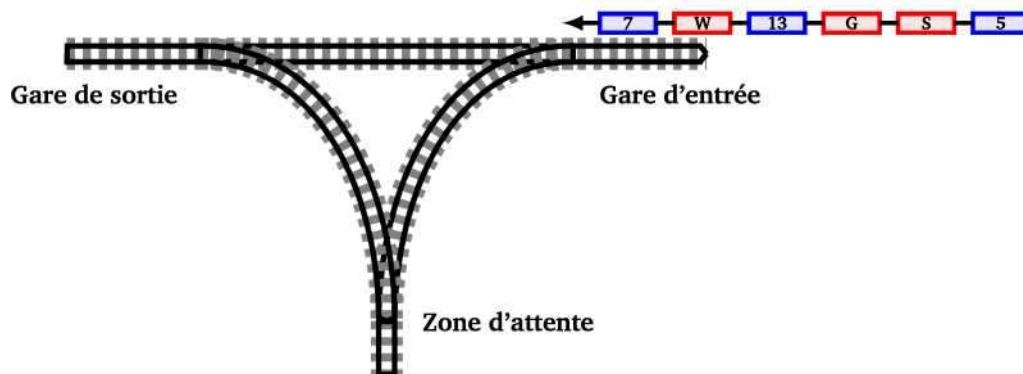
Un train comporte des wagons bleus qui portent un numéro et des wagons rouges qui portent une lettre.



Le chef de gare souhaite séparer les wagons : d'abord tous les bleus et ensuite tous les rouges (l'ordre des wagons bleus n'a pas d'importance, l'ordre des wagons rouges non plus).

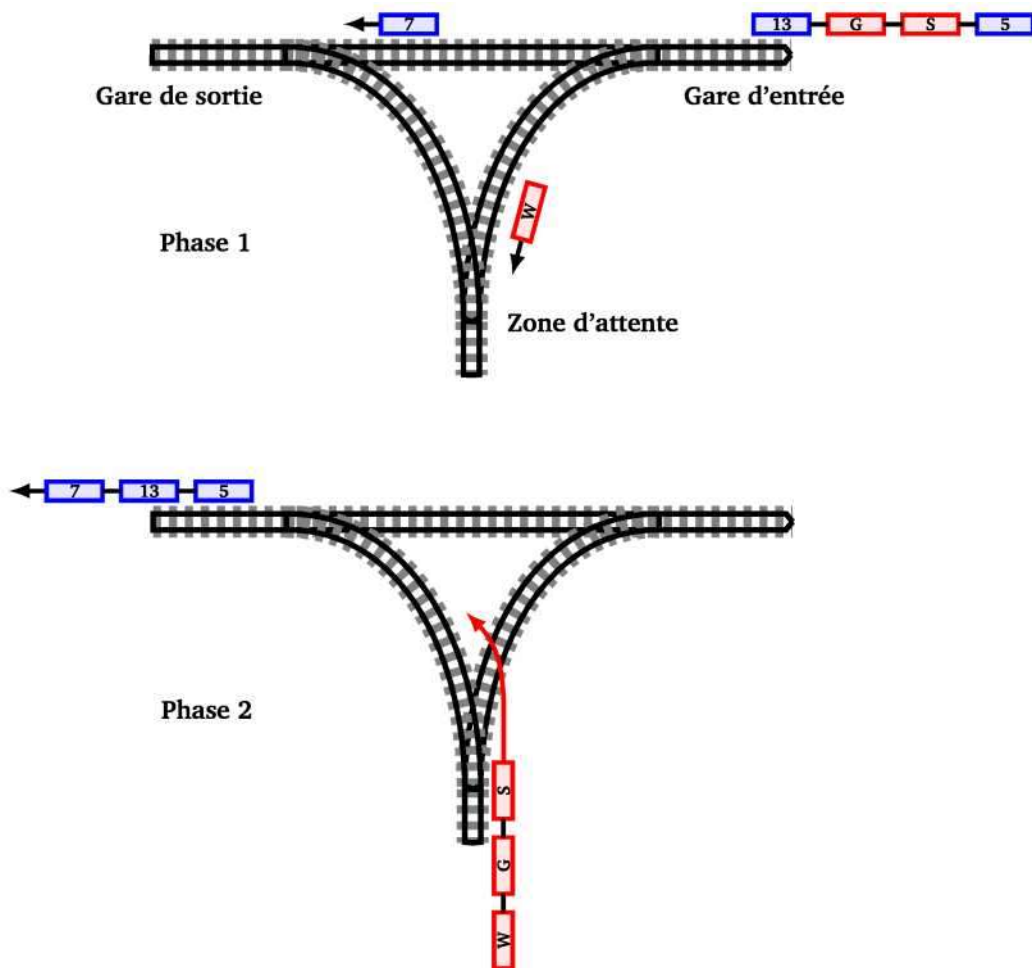


Pour cela, il dispose d'une gare de sortie et d'une zone d'attente : un wagon peut soit être directement envoyé à la gare de sortie, soit être momentanément stocké dans la zone d'attente.



Voici les instructions du chef de gare.

- **Phase 1.** Pour chaque wagon du train :
 - si c'est un wagon bleu, envoyez-le directement en gare de sortie ;
 - si c'est un wagon rouge, envoyez-le dans la zone d'attente.
 -
- **Phase 2.** Ensuite, déplacez un par un les wagons (rouges) de la zone d'attente vers la gare de sortie en les raccrochant aux autres.



Voici comment nous allons modéliser le train et son triage.

- Le train est une chaîne de caractères formée d'une suite de nombres (les wagons bleus) et de lettres (les wagons rouges) séparés par des espaces. Par exemple `train = "G 6 Z J 14"`.
- On obtient la liste des wagons par la commande `train.split()`.
- On teste si un wagon est bleu en regardant s'il est marqué d'un nombre, par l'instruction `wagon.isdigit()`.
- Le train reconstitué par les wagons triés est aussi une chaîne de caractères. Au départ, c'est la chaîne vide.
- La zone d'attente sera la pile. Au départ, la pile est vide. On va y ajouter uniquement les wagons rouges. À la fin, on vide la pile vers la queue du train reconstitué.

En suivant les instructions du chef de gare et en utilisant les opérations sur une pile, écrire une fonction `tri_wagons` qui sépare les wagons bleus et rouges d'un train.

Les spécifications de la fonction `tri_wagons` sont données dans l'encadré ci-dessous.

`tri_wagons()`

Usage : `tri_wagons(train)`

Entrée : une chaîne de caractères avec des wagons bleus (nombres) et des wagons rouges (lettres)

Sortie : les wagons bleus d'abord et les rouges ensuite

Action : utilise la pile

Exemple :

- `tri_wagons("A 4 C 12")` renvoie `"4 12 C A"`
- `tri_wagons("K 8 P 17 L B R 3 10 2 N")` renvoie `"8 17 3 10 2 N R B L P K"`

Cours 4 (Notation polonaise).

Réécriture en notation polonaise (de son vrai nom, notation polonaise inverse) est une autre façon d'écrire une expression algébrique. Son avantage est que cette notation n'utilise pas de parenthèses et qu'elle est plus facile à manipuler pour un ordinateur. Son inconvénient est que nous n'y sommes pas habitués.

Voici la façon classique d'écrire une expression algébrique (à gauche) et son écriture polonaise (à droite).

Dans tous les cas, le résultat sera 13 !

Classique : $7 + 6$

Polonaise : $7\ 6\ +$

Autres exemples :

- classique : $(10 + 5) \times 3$; polonaise : $10\ 5\ +\ 3\ \times$
- classique : $10 + 2 \times 3$; polonaise : $10\ 2\ 3\ \times\ +$
- classique : $(2 + 8) \times (6 + 11)$; polonaise : $2\ 8\ +\ 6\ 11\ +\ \times$

Voyons comment calculer la valeur d'une expression en écriture polonaise.

- On lit l'expression de gauche à droite :

$2\ 8\ +\ 6\ 11\ +\ \times$


- Lorsque l'on rencontre un premier opérateur (+, ×, ...) on calcule l'opération avec les deux membres juste avant cet opérateur :

$\underbrace{2\ 8\ +}_{2+8}\ 6\ 11\ +\ \times$

- On remplace cette opération par le résultat :

$\underbrace{10}_{\text{résultat de } 2+8}\ 6\ 11\ +\ \times$

- On continue la lecture de l'expression (on cherche le premier opérateur et les deux termes juste avant) :

$10\ \underbrace{6\ 11\ +}_{6+11=17}\ \times$ devient $10\ 17\ \times$ qui vaut 170

- À la fin il ne reste qu'une valeur, c'est le résultat ! (Ici 170.)

Autres exemples :

- $8\ 2\ \div\ 3\ \times\ 7\ +$

$\underbrace{8\ 2\ \div}_{8\div 2=4}\ 3\ \times\ 7\ +$ devient $\underbrace{4\ 3\ \times}_{4\times 3=12}\ 7\ +$ devient $12\ 7\ +$ qui vaut 19

- $11\ 9\ 4\ 3\ +\ -\ \times$

$11\ 9\ \underbrace{4\ 3\ +}_{4+3=7}\ -\ \times$ devient $11\ \underbrace{9\ 7\ -}_{9-7=2}\ \times$ devient $11\ 2\ \times$ qui vaut 22

Exercice.

Détails et résultats à compléter dans le notebook, dans la cellule dédiée.

Calculer la valeur des expressions suivantes :

- $13\ 5\ +\ 3\ \times$
- $3\ 5\ 7\ \times\ +$
- $3\ 5\ 7\ +\ \times$
- $15\ 5\ +\ 4\ 12\ +\ \times$

Activité 3 (Calculatrice polonaise).

Objectifs : programmer une mini-calculatrice qui calcule les expressions en écriture polonaise.

1. Écrire une fonction `operation` qui calcule la somme ou le produit de deux nombres.
Les spécifications de la fonction `operation` sont données dans l'encadré ci-dessous.

operation()

Usage : `operation(a,b,op)`
Entrée : deux nombres `a` et `b`, un caractère d'opération `"+"` ou `"*"`
Sortie : le résultat de l'opération `a + b` ou `a * b`
Exemple :

- `operation(2,4,"+")` renvoie 6
- `operation(2,4,"*")` renvoie 8

2. Programmer une calculatrice polonaise, selon l'algorithme suivant :

Algorithme.

- Entrée : une expression en écriture polonaise (une chaîne de caractères).
 - Sortie : la valeur de cette expression.
 - Exemple : `"2 3 + 4 *"` (le calcul $(2 + 3) \times 4$ donne 20).
- Partir avec une pile vide.
- Pour chaque élément de l'expression (lue de gauche à droite) :
 - si l'élément est un nombre, alors ajouter ce nombre à la pile,
 - si l'élément est une opération, alors :
 - dépiler une fois pour obtenir un nombre b ,
 - dépiler une seconde fois pour obtenir un nombre a ,
 - calculer $a + b$ ou $a \times b$ selon l'opération,
 - ajouter ce résultat à la pile.
- À la fin, la pile ne contient qu'un seul élément, c'est le résultat du calcul.

calculatrice_polonaise()

Usage : `calculatrice_polonaise(expression)`
Entrée : une expression en notation polonaise (chaîne de caractères)
Sortie : le résultat du calcul
Action : utilise une pile
Exemple :

- `calculatrice_polonaise("2 3 4 + +")` renvoie 9
- `calculatrice_polonaise("2 3 + 5 *")` renvoie 25

3. Modifier le code pour prendre en charge la soustraction et la division !

Activité 5 (Expression bien parenthésée).

Objectif : déterminer si les parenthèses d'une expression sont placées de façon cohérente.

Voici des exemples d'expressions bien et mal parenthésées :

- $2 + (3 + b) \times (5 + (a - 4))$ est correctement parenthésée ;
- $(a + 8) \times 3 + 4$ est mal parenthésée : il y a une parenthèse fermante «) » seule ;
- $(b + 8/5) + (4$ est mal parenthésée : il y a autant de parenthèses ouvrantes « (» que de parenthèses fermantes «) » mais elles sont mal positionnées.

1. Ecrire une fonction `parentheses_correctes` qui prend pour paramètre `expression`, une chaîne de caractères pouvant représenter une expression mathématique, et qui renvoie `True` si l'expression est bien parenthésée, `False` sinon.

Les spécifications de la fonction `parentheses_correctes` sont données dans l'encadré ci-dessous.

parentheses_correctes()

Usage : `parentheses_correctes(expression)`
Entrée : une expression (chaîne de caractères)
Sortie : vrai ou faux selon que les parenthèses sont correctes ou pas
Action : utilise une pile

Exemple :

- `parentheses_correctes("(2+3)*(4+(8/2))")` renvoie `True`
- `parentheses_correctes("(x+y)*((7+z)")` renvoie `False`

2. Améliorer cette fonction en une autre fonction nommée `crochets_parentheses_correctes` pour tester une expression avec des parenthèses et des crochets.
 - voici une expression correctes : $[(a + b) * (a - b)]$;
 - voici des expressions non correctes : $[a + b), (a + b) * [a - b)$.

Cette fois la pile peut donc contenir des parenthèses ouvrantes " (" ou bien des crochets ouvrants " [". Chaque fois que l'on trouve une parenthèse fermante ") " dans l'expression, il faut que le haut de la pile soit une parenthèse ouvrante " (". Chaque fois que l'on trouve un crochet fermant "] " dans l'expression, il faut que le haut de la pile soit un crochet ouvrant " [".

Activité 5 (Conversion en écriture polonaise).

Objectif : transformer une expression algébrique classique avec parenthèses en une écriture polonaise. L'algorithme est une version très améliorée de l'activité précédente.

Tu es habitué à l'écriture $(13 + 5) \times 7$; tu as vu que l'ordinateur savait facilement calculer $13 + 5 \times 7$. Il ne reste plus qu'à passer de l'expression algébrique classique (avec parenthèses) à l'écriture polonaise (sans parenthèses) !

Voici l'algorithme pour des expressions ne comportant que des additions et des multiplications.

Algorithme.

Entrée : une expression en écriture habituelle

Sortie : l'expression écrite en notation polonaise

- Partir avec une pile vide.
- Partir avec une chaîne vide polonaise qui à la fin contiendra le résultat.
- Pour chaque caractère de l'expression (lue de gauche à droite) :
 - **si** le caractère est un nombre, alors ajouter ce nombre à la chaîne de sortie polonaise ;
 - **si** le caractère est une parenthèse ouvrante " (", alors ajouter ce caractère à la pile ;
 - **si** le caractère est l'opérateur de multiplication " * ", alors ajouter ce caractère à la pile ;
 - **si** le caractère est l'opérateur d'addition " + ", alors
 - tant que** la pile n'est pas vide :
 - dépiler un élément
 - si** cet élément est l'opérateur de multiplication alors :
 - ajouter cet élément à la chaîne de sortie polonaise
 - sinon** :
 - si** cet élément est une parenthèse ouvrante " (", alors
 - empiler cet élément (on le remet sur la pile après l'avoir enlevé)
 - sinon**
 - ajouter cet élément à la chaîne de sortie polonaise
 - sortir de la boucle « tant que »
 - enfin, ajouter l'opérateur d'addition " + " à la pile.
 - **si** le caractère est une parenthèse fermante ") ", alors
 - tant que** la pile n'est pas vide :
 - dépiler un élément,
 - si** cet élément est une parenthèse ouvrante " (", alors :
 - sortir de la boucle « tant que »
 - sinon** :
 - ajouter cet élément à la chaîne de sortie polonaise

écriture_polonaise()

Usage : `écriture_polonaise(expression)`

Entrée : une expression classique (avec les éléments séparés par des espaces)

Sortie : l'expression en notation polonaise

Action : utilise la pile

Exemple :

- `écriture_polonaise("2 + 3")` renvoie "2 3 +"
- `écriture_polonaise("4 * ( 2 + 3 )")` renvoie "4 2 3 + *"
- `écriture_polonaise("( 2 + 3 ) * ( 4 + 8 )")` renvoie "2 3 + 4 8 + *"

Dans cet algorithme, on appelle abusivement « caractère » d'une expression chaque élément entre deux espaces. Exemple : les caractères de "(17 + 10) * 3" sont (,17,+,10,)* et 3.

Tu vois que l'addition a un traitement plus compliqué que la multiplication. C'est dû au fait que la multiplication est prioritaire devant l'addition. Par exemple $2 + 3 \times 5$ signifie $2 + (3 \times 5)$ et pas $(2 + 3) \times 5$.

Fin du TP :

Modifier le code de l'activité 6 pour prendre en charge la soustraction et la division !

Terminer ce TP en vérifiant que tout fonctionne correctement avec différentes expressions.

Par exemple :

- Définir l'expression `exp = "( 17 * ( 2 + 3 ) ) + ( 4 + ( 8 * 5 ) )"`
- Demander à Python d'évaluer cette expression avec l'instruction `eval(exp)`.
- L'instruction `calculatrice_polonaise(ecriture_polonaise(exp))` doit renvoyer le même résultat.
- Tester sur d'autres expressions en veillant à faire intervenir les 4 opérateurs.

*** FIN DU TP ***